SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

*I _**SHAIK THOUHID (192421280)**_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Thouhid

Signature of the student:

Scenario 1: AI Drones for Emergency Medicine Delivery in a Flood Zone

Problem Formulation

Component	Definition
State	Current position of the drone (a waypoint on a graph or grid of the flood region)
Initial state	Launch base / depot
Goal state	Medical delivery point
Actions	Move to an adjacent waypoint that is not currently reported as blocked
Step cost c(n, n')	Depends on terrain, weather (wind, rain), and risk; it varies over time
Heuristic h(n)	Straight-line (Euclidean) distance from n to the goal
Percepts	Partial real-time updates on blocked paths

The drone must minimise delay and risk. Because the environment is dynamic and only partly known, the search has to be fast, safe, and able to re-plan.

(i) Greedy Best-First Search (GBFS) in This Scenario

Behaviour. GBFS expands the node that looks closest to the goal, using only the heuristic: **f(n) = h(n)**. The drone always heads towards the waypoint with the smallest straight-line distance to the delivery point. It keeps a priority queue ordered by h(n) and ignores the cost already spent (g(n)).

In the flood region this means the drone flies as directly as possible towards the hospital or relief camp. If the direct line is clear, it reaches the goal very quickly.

Strengths under uncertainty

- Very fast decisions: few nodes are expanded, so a route appears almost immediately. This suits an urgent mission.
- Low memory and computation, which is useful on a drone's limited onboard hardware.
- Re-planning after a blocked path is cheap because the search is small.

Weaknesses under uncertainty

- It ignores movement cost. A short straight route through a storm zone or rough terrain looks attractive although it is slow and dangerous.
- It is **not optimal**. The path found can be much costlier than the best path.
- It can be trapped by dead ends. If a flooded river or collapsed road lies between the drone and the goal, h(n) keeps decreasing towards the obstacle and the drone wastes time before backing out.
- It can be incomplete when repeated states are not handled (it may loop).
- It trusts the heuristic blindly. Straight-line distance says nothing about blocked roads, so under uncertainty its decisions are easily misled.

(ii) A* in the Same Situation

Behaviour. A* evaluates each node with

f(n) = g(n) + h(n)

where g(n) is the actual cost from the start to n (terrain and weather aware) and h(n) is the estimated cost from n to the goal.

How it balances cost and heuristic

- g(n) keeps the search honest. It penalises routes that have already been expensive, for example those that crossed high-wind areas.
- h(n) gives direction and pulls the search towards the goal, so A* does not explore blindly like uniform-cost search.
- When the heuristic is **admissible** (never overestimates the true cost) and **consistent**, A* is both **complete and optimal**. For the drone, h must be scaled so that it never exceeds the true minimum cost per unit distance (for example, use the best-case speed and calm-weather cost).
- If the drone meets a high-cost region, g(n) rises quickly, f(n) of those nodes grows, and A* automatically switches to cheaper branches around the storm or damaged terrain.

Limitations. A* expands more nodes than GBFS and needs more memory. On a very large map this can delay the first move. Variants such as **Weighted A*** ($f = g + w \cdot h$, $w > 1$) trade a bounded loss of optimality for speed.

(iii) Impact of Heuristic Accuracy

Heuristic quality	Effect on GBFS	Effect on A*
Perfect (h = true cost)	Goes straight to the goal	Expands only nodes on the optimal path

Heuristic quality	Effect on GBFS	Effect on A*
Accurate but slightly low (admissible)	Fast, but still ignores cost, so may not be optimal	Optimal, expands few nodes
Very low or zero (h = 0)	Becomes random or blind	Reduces to Dijkstra / uniform-cost search: optimal but slow
Overestimating (inadmissible)	Little change; it never promised optimality	May return a suboptimal path (it can skip the best route)
Misleading (ignores blocked roads or weather)	Severe: drone is drawn into dead ends and bad weather	Degraded efficiency, but g(n) corrects errors, so path quality is more robust

Key points

- GBFS depends **entirely** on the heuristic, so an inaccurate one hurts both speed and path quality.
- A* uses the heuristic only as guidance, while the real cost g(n) supplies correction. Poor accuracy mostly increases the number of expanded nodes rather than breaking correctness (as long as h stays admissible).
- In this scenario, straight-line distance is weak because rivers, blocked roads, and wind make real cost much larger. A better heuristic is a **weather-adjusted distance** or a precomputed cost map from past flights.

(iv) Effect of Dynamic Changes (Blocked Paths)

- A path that was valid when planned can become invalid mid-flight, so the stored plan is no longer reliable.
- **GBFS** recovers quickly because it only needs a new greedy choice, but it may repeatedly walk into new blockages and fail to find a safe detour.
- **A*** must recompute its search tree when edge costs change or edges disappear. Re-running it from scratch is costly if blockages are frequent. A* does give a **cost-aware** detour once it replans.
- Partial information means the drone does not know all blocked paths. A common approach is the **free-space assumption**: treat unknown edges as open, plan, and replan when a blockage is sensed.
- **Incremental replanning algorithms** (D* Lite, LPA*, Anytime Repairing A*) reuse earlier search effort and update only the affected part of the search graph. They keep A*'s optimality while reducing replanning delay.

- Search performance falls as the rate of change increases: more replans, more wasted flight, and higher energy use.

(v) Recommended Algorithm and Justification

Recommendation: A (in practice Weighted A with incremental replanning such as D* Lite), using a risk-aware cost function and a weather-adjusted heuristic.**

Criterion	Justification
Real-time constraints	Weighted A* with a modest weight (for example 1.2 to 1.5) returns a good route quickly; D* Lite repairs the plan in milliseconds after a blockage.
Optimality	With an admissible heuristic, A* finds the minimum-cost route in delay and energy; weighting gives bounded suboptimality.
Safety	Risk (strong wind, rough terrain, near-blocked zones) is added to g(n), so dangerous areas are avoided even if they are shorter. GBFS cannot do this.
Robustness under uncertainty	The real-cost term corrects heuristic errors, and replanning handles new blockages.

GBFS can be used only as a quick fallback when time is extremely short, but because it ignores cost and risk it is not suitable as the main method for life-critical delivery.

Scenario 2: AI Traffic Signal Optimisation in a Smart City

Problem Formulation as an Optimisation Problem

- **Decision variables.** For each intersection i: cycle length C_i, green time split for each phase g_ij, and offset θ_i (relative to neighbouring signals). A complete solution is the vector of all timings for the hundreds of intersections.
- **State space.** Every possible assignment of timings. With N intersections and k candidate settings each, there are k^N states (for example, 10^300 for 300 intersections and 10 settings).
- **Objective function (to minimise):**

$J = w_1 \cdot$(Total waiting time) + $w_2 \cdot$(Fuel consumption) + $w_3 \cdot$(Congestion index)

where w_1, w_2, w_3 are weights set by city policy. The values are estimated with a traffic simulator or live sensor data.

- **Constraints.** Minimum and maximum green time, pedestrian crossing time, emergency vehicle priority, and safety clearance between conflicting phases.
- **Neighbourhood (moves).** Change one intersection's green split or offset by a small amount.

Why Traditional Search Is Inefficient

- **Exponential state space.** BFS, DFS, and uniform-cost search cannot enumerate k^N states.
- **No clear goal state.** The aim is "as good as possible", not reaching one particular goal, so goal-based search does not apply directly.
- **No reliable admissible heuristic.** A* needs a heuristic estimating remaining cost, which is hard to define for interacting signals.
- **Costly evaluation.** Each candidate needs a traffic simulation, so even a few thousand evaluations are expensive.
- **Dynamic environment.** Demand changes with time of day, accidents, and events, so a stored solution becomes outdated and a full search would have to be repeated again and again.
- **Strong interaction.** Changing one signal affects neighbouring intersections, so the problem cannot be solved independently piece by piece.

This is why **local search and meta-heuristics**, which keep only the current state and improve it, are preferred.

(i) Hill Climbing: How It Works and Its Limitations

Working

1. Start with the current signal timing plan (or a random one).
2. Generate neighbours, for example shifting 5 seconds of green from one phase to another at one intersection.
3. Evaluate J for each neighbour (using the simulator or sensors).
4. Move to the best neighbour if it improves J (steepest-ascent). Variants: first-choice, stochastic, and random-restart hill climbing.
5. Stop when no neighbour is better.

Advantages. Simple, very low memory, fast, and suitable for continuous adjustment of a running system.

Limitations

- Gets stuck at local optima.
- Stalls on plateaus where neighbours have equal value.
- Cannot follow narrow ridges, where improving one parameter requires changing several together.
- Result depends heavily on the starting point.
- Cannot accept a temporary worsening, so it never escapes a poor region.

(ii) Local Maxima, Plateaus, and Ridges: Real-World Interpretation

Issue	Meaning in the algorithm	Traffic interpretation
Local maximum (local optimum)	A state better than all its neighbours but worse than the global best	A set of timings in which each junction looks tuned and no single change helps, yet a different overall coordination (a "green wave" along a whole corridor) would cut waiting time much more.
Plateau	A flat region where neighbouring states have the same value	In light traffic or at night, small timing tweaks change nothing measurable, so the algorithm has no signal about which way to move and may wander or stop.
Ridge	A sequence of local peaks that cannot be climbed with single-variable moves	To improve flow on an avenue, several consecutive signal offsets must move **together**. Changing one signal alone worsens the result, so hill climbing rejects every move.

(iii) How Simulated Annealing and Genetic Algorithms Overcome These Limitations

Simulated Annealing (SA)

- Inspired by the cooling of metal. A random neighbour is picked; if it is better it is accepted; if it is worse it is still accepted with probability **P = e^(−ΔE / T)**, where ΔE is the increase in cost and T is the temperature.
- At high T the search explores widely (escapes local optima and crosses plateaus and ridges). As T is lowered under a cooling schedule, it behaves more like hill climbing and settles into a good solution.
- It is simple, needs little memory, and can be warm-started from the current signal plan.

Genetic Algorithms (GA)

- Each **chromosome** encodes a complete timing plan (all cycle lengths, splits, and offsets).
- **Fitness** = 1 / J (better plans score higher).
- **Selection** picks good plans, **crossover** combines timings from two plans (for example the corridor from one and the district from another), and **mutation** randomly modifies timings to add diversity.
- Because a **population** of solutions is evolved in parallel, GA explores many regions at once, handles ridges through crossover, and is easy to parallelise on many processors.

Other helpful ideas. Random-restart hill climbing and local beam search also reduce local-optimum trapping, but SA and GA do so in a more principled way.

(iv) Recommended Approach and Justification

Recommendation: a hybrid (memetic) approach: Genetic Algorithm for global exploration, with Simulated Annealing or hill climbing for fine local adjustment, applied hierarchically (corridor or district level) and warm-started from the current plan.

Criterion	Justification
Scalability	GA's population evaluations run in parallel; splitting the city into zones or corridors reduces each subproblem's size. SA and hill climbing need only the current state, so memory stays small.
Adaptability	When traffic changes, the previous best population or plan is used as the starting point, so the system re-optimises quickly instead of starting over. Online hill climbing or SA can make small, safe adjustments every few minutes.
Solution quality	Global search by GA avoids local maxima; local refinement polishes the result.
Safety and stability	Hard constraints (minimum green, pedestrian time) are enforced during moves, and changes are applied gradually to avoid sudden disruption.

Pure hill climbing is too easily trapped, and pure GA may be slow for real-time updates; the hybrid gives both quality and responsiveness.

Scenario 3: Autonomous Mars Rover in Unknown Terrain

(i) Why an Online Search Agent Is Needed Instead of Offline Search

- **Offline search** computes a complete plan before acting. It needs a full model of the environment (map, action results, costs). The rover has no complete map of Mars.
- **Online search** interleaves *planning and acting*: the agent chooses an action, executes it, observes the result, and only then decides the next action. It is the right choice when the environment is unknown or dynamic.
- The rover can learn the result of an action (for example, which cells are safe next to it) only **after** it moves and senses.
- Communication delay with Earth (several minutes to over 20 minutes one way) means that remote control is impossible for real-time decisions; the rover must decide on its own.
- Mistakes are costly and often irreversible (falling into a crater, getting stuck in sand), so the agent must act carefully and keep updating its knowledge.
- Limited onboard energy and computing power make it impractical to compute plans over an unknown full planet-scale map.

The online agent knows only: the actions available in its current state, the cost of a step once it has been taken, and whether the current state is a goal (for example, a sample site).

(ii) Challenges of Partial Observability and Dynamic Environments

Partial observability

- Sensors (cameras, LIDAR, hazard detectors) have a limited range and field of view, so only the nearby surroundings are visible.
- Sensor noise and shadows can hide rocks or crater edges and create false readings.
- Localisation uncertainty: wheel slip and drift make the rover unsure of its exact position, so it must reason about a **belief state** (a set or probability distribution of possible positions) rather than a single known state.
- Hidden hazards, such as loose soil under a flat-looking surface, cannot be detected from a distance.

Dynamic environment

- Dust storms, changing light and temperature affect sensing and energy.
- Terrain can change (shifting sand, rolling rocks), so earlier information may become outdated.
- Energy, battery, and time limits keep changing as the rover moves.

Consequences. The agent cannot guarantee a globally optimal route, must tolerate wrong or outdated information, must avoid dead ends it cannot leave, and must prefer **safe** over **fast** actions.

(iii) How the Rover Builds and Updates Its Internal Model

The rover runs a repeated **sense, update, plan, act** loop:

1. **Sense.** Collect data from cameras, LIDAR, inertial sensors, and wheel encoders.
2. **Update the map.** Maintain an occupancy grid or terrain-cost map in which each cell holds a probability of being free, rock, crater, or unknown. The probabilities are revised using Bayesian updating as new observations arrive.
3. **Localise.** Use SLAM (simultaneous localisation and mapping) to correct the position estimate using landmarks.
4. **Record experience.** Store a table of results *result(s, a)* (which state an action led to), visited states, and learned costs. Costs of risky cells are increased after bad experiences, for example slipping.
5. **Plan locally.** Search on the current model for the next safe steps towards a goal or toward an unexplored frontier.
6. **Act and repeat.**

A classic learning rule used by online search is **LRTA* (Learning Real-Time A*)**: the agent keeps an estimate H(s) of the cost to the goal for each visited state and updates it with

H(s) = min over actions a of [c(s, a, s') + H(s')]

so that heuristic values improve every time the rover visits a state, which stops it from looping and helps it escape local traps.

(iv) Comparison: Online Search vs Uninformed and Informed Search

Aspect	Uninformed search (BFS, DFS, UCS)	Informed search (A*, Greedy)	Online search (online DFS, LRTA*, D* Lite)
Knowledge needed	Complete model of states and actions	Complete model plus heuristic	Only local knowledge; learns while moving

Aspect	Uninformed search (BFS, DFS, UCS)	Informed search (A*, Greedy)	Online search (online DFS, LRTA*, D* Lite)
Planning style	Offline: plans fully, then acts	Offline: plans fully, then acts	Interleaves planning and acting
Handles unknown map	No	No (unless the map is already given)	Yes
Handles changes	Must restart the search	Must replan from scratch (unless incremental)	Updates the model and continues
Efficiency	Explores blindly, high cost	Efficient with a good heuristic	Efficient in exploration; may revisit states
Optimality	BFS and UCS optimal on a known graph	A* optimal with admissible heuristic	Not guaranteed optimal, but complete in safely explorable spaces
Safety in unknown terrain	Cannot assess unseen risk	Same limitation	Can add caution (risk costs, return paths)
Memory	High	Medium to high	Low to medium (keeps a table of visited states)

Notes. Online DFS needs actions to be reversible (so the agent can backtrack), which is not always true on Mars. Informed algorithms such as A* can still be used *inside* an online agent for local planning on the currently known map.

(v) Most Suitable Approach and Justification

Recommendation: an online search agent that combines LRTA*-style learning or D* Lite re-planning with a risk-aware cost map and frontier-based exploration.

- **Uncertainty.** It makes decisions using only what has been sensed and uses probabilistic maps and belief states.
- **Adaptability.** The model is updated after each step. With D* Lite, only the part of the plan affected by new information is recalculated, so replanning is fast.

- **Safety.** Cells with craters, steep slopes, or uncertain readings receive high cost or are forbidden; the rover keeps an escape path to known safe ground, and moves slowly into unknown cells.
- **Efficiency.** Local A* planning over the known map keeps routes short while the learning rule prevents repeated visits to the same dead end.
- **Autonomy.** It works without constant instructions from Earth, which fits the communication delay.

Pure offline A* or uninformed search cannot be used because the map does not exist in advance.

Scenario 4: University Exam Timetable as a Constraint Satisfaction Problem

CSP Formulation: Variables, Domains, and Constraints

A CSP is defined by a set of variables X, a domain D for each variable, and a set of constraints C. A solution is a complete assignment that satisfies every constraint.

Variables

Variable	Meaning
T(e)	Time slot (day and session) of exam e, for every course exam e
R(e)	Exam hall assigned to exam e
F(e)	Invigilator / faculty member supervising exam e

Domains

Variable	Domain
T(e)	{1, 2, ..., S}, all available exam slots (for example 10 days × 2 sessions = 20 slots)
R(e)	{H1, H2, ..., Hm}, the available exam halls
F(e)	The set of faculty members qualified and free to supervise exam e

Constraints

#	Requirement	Constraint (type)	Explanation
1	No student has	T(a) ≠ T(b) for every pair of exams a,	Two courses with a common

#	Requirement	Constraint (type)	Explanation
	overlapping exams	b that share at least one student (binary)	student cannot sit in the same slot.
2	Limited number of halls	If T(a) = T(b) then R(a) ≠ R(b); and capacity(R(e)) ≥ enrolment(e) (binary and unary)	One hall holds only one exam per slot and must be large enough. Equivalent to *all-different* over (slot, hall) pairs.
3	Some subjects before others	T(a) < T(b) when a is a prerequisite of b (binary precedence)	Orders exams according to the syllabus or policy.
4	Faculty availability	T(e) ∈ available slots of F(e) (unary); and if T(a) = T(b) then F(a) ≠ F(b) (binary)	A faculty member can supervise only in free slots and only one exam at a time.
5	Special accommodations	For accommodated students, R(e) ∈ accessible or private halls and the slot must allow extra time (for example no exam right after it in the same hall) (unary and binary)	Respects needs such as extra time, accessible rooms, or separate rooms.
6	Optional soft preferences	Avoid more than 2 exams for a student in one day; spread exams evenly (soft)	Not mandatory; they improve quality. They can be handled as costs to minimise.

The **constraint graph** has one node per exam; an edge links two exams if they share students, a hall, a faculty member, or a precedence relation. Dense connections show how the problem becomes harder as the number of courses and students grows (the core of the problem is similar to **graph colouring**, which is NP-hard).

(ii) How Backtracking Search Works

Backtracking search is depth-first search that assigns **one variable at a time** and checks constraints after every assignment.

1. Pick an unassigned variable (for example exam e).
2. Choose a value from its domain (a slot and hall).

3. If the value violates a constraint with the assignments made so far, try the next value.
4. If consistent, move on recursively to the next variable.
5. If no value of a variable works, **backtrack** to the previous variable and try another value for it.
6. Stop when all variables are assigned (solution) or all options are exhausted (no solution).

Example. Assign Maths to slot 1, Hall H1. Physics shares students with Maths, so slot 1 is rejected for Physics and it receives slot 2. If later Chemistry has no feasible slot because of faculty and hall limits, the search backtracks and changes Physics or Maths.

Improvements that make backtracking practical

- **MRV (minimum remaining values):** choose the exam with the fewest legal slots left (the most constrained).
- **Degree heuristic:** choose the exam involved in the most constraints (such as a course with many shared students).
- **LCV (least constraining value):** try first the slot that leaves the most options for neighbours.
- Plain backtracking can waste enormous time repeating the same failures (thrashing) when a bad early choice is only discovered much later.

(iii) How Constraint Propagation (Forward Checking) Improves Efficiency

Forward checking. Whenever a variable is assigned, remove from the domains of its unassigned neighbours all values that are now inconsistent. If any neighbour's domain becomes **empty**, backtrack immediately instead of continuing.

Example. After Maths is placed in slot 1, forward checking removes slot 1 from the domain of every exam that shares a student with Maths, and removes H1 from the hall options of other exams in slot 1. If an exam is left with no slot at all, the algorithm detects the failure now, not after dozens more assignments.

Benefits

- Detects dead ends **earlier**, which prunes large parts of the search tree.
- Provides accurate domain sizes for the MRV heuristic.
- Adds little overhead compared with the work it saves.

Stronger propagation. *Arc consistency* (AC-3) and *Maintaining Arc Consistency (MAC)* also check indirect effects: if removing a value from one exam's domain leaves a neighbour without support, that neighbour's value is removed too, and so on. This catches problems forward checking misses, such as

precedence chains (A before B before C) that cannot fit into the remaining slots. It costs more per step but often reduces the search drastically on tightly constrained timetables.

(iv) Real-World Challenges and Best Strategy for Scalability

Challenges

- **Scale.** Thousands of exams, tens of thousands of students, and many halls give a huge search space.
- **Over-constrained cases.** A perfect solution may not exist; some preferences must be relaxed.
- **Hard and soft constraints.** Mandatory rules (no clashes) must be separated from preferences (fair spacing).
- **Late changes.** Faculty leave, new enrolments, and hall closures require quick updating without rebuilding the whole timetable.
- **Fairness and special needs.** Accommodations and student well-being (not too many exams in a row) must be handled consistently.
- **Data quality.** Enrolment or availability errors lead to invalid timetables.

Recommended strategy: a hybrid CSP approach

1. Model hard constraints as a CSP and reduce domains first with **node and arc consistency** preprocessing.
2. Solve with **backtracking + MRV + degree + LCV + forward checking / MAC**.
3. Break the problem into smaller parts (by faculty, department, or day) using the structure of the constraint graph, then merge the partial solutions.
4. Treat soft constraints as an objective (weighted penalties) and improve the first feasible timetable using **local search such as min-conflicts**.
5. Use min-conflicts or incremental re-solving for **last-minute changes**, so only the affected exams are moved.

Justification. Systematic backtracking with propagation guarantees that all hard constraints are met and finds a solution if one exists. Heuristics and decomposition make it scale, while local search gives quick repair and flexibility for changing conditions and preferences.

Scenario 5: Strategic Game AI for Real-Time Decision Making

(i) How Minimax Models the Problem

Minimax models the game as a **two-player, zero-sum adversarial search**:

- **MAX** is the AI opponent, which tries to maximise its utility.
- **MIN** is the human player, assumed to play the moves that are worst for the AI (minimise the AI's utility).
- **Game tree:** each node is a game state; each edge is a legal move (build, attack, move units, research); levels alternate between MAX and MIN.
- **Utility / evaluation values** are placed at the leaves (win, loss, or an estimated score at the depth limit).
- **Backing up values:** a MAX node takes the **maximum** of its children's values and a MIN node takes the **minimum**. The move at the root with the highest backed-up value is chosen.

MINIMAX(s) = Utility(s) if s is terminal or at the depth cutoff; max over actions of MINIMAX(Result(s, a)) if it is MAX's turn; min over actions of MINIMAX(Result(s, a)) if it is MIN's turn.

Minimax assumes the opponent plays optimally, so it gives a **worst-case guaranteed** decision. In a real-time strategy game the continuous play is approximated by dividing time into short decision steps and grouping low-level actions into **macro actions** (for example "attack the north gate with 10 units") so that a tree can be built.

(ii) Computational Challenges of Large Game Trees

- **Exponential growth.** If the branching factor is b and the search depth is d, minimax examines about **b^d** nodes. Time is O(b^d) and space is O(b·d).
- **Example.** With b = 30 moves and d = 10 plies, there are about 30^10 ≈ 5.9 × 10^14 nodes, which is impossible within a fraction of a second.
- **Strict time limits.** The AI must respond in real time; a late move is as bad as a wrong move.
- **Very large branching.** Many units, each with several actions, plus simultaneous moves, produce enormous numbers of joint moves.
- **Imperfect information and chance.** Fog of war and random combat outcomes make the pure deterministic perfect-information tree an approximation.
- **Evaluation quality.** Because the tree must be cut off early, the result depends on how well the evaluation function judges incomplete positions. Poor evaluation leads to the *horizon effect* (the AI misses a danger or gain just beyond the search depth).

- **Repeated states.** The same position can be reached by different move orders, wasting effort unless stored.

(iii) How Alpha-Beta Pruning Improves Efficiency

Idea. Alpha-beta pruning returns **exactly the same decision as minimax** but skips branches that cannot influence the final choice. It keeps two bounds during the depth-first search:

- **α (alpha):** the best (highest) value that MAX can already guarantee along the current path.
- **β (beta):** the best (lowest) value that MIN can already guarantee along the current path.

Pruning rule: whenever **α ≥ β** at a node, the remaining children of that node are not examined, because the opponent would never allow the game to reach it.

Detailed reasoning (small example)

1. The root is a MAX node with two MIN children, A and B.
2. Child A is explored fully and its value is 6, so MAX now knows it can get at least 6: **α = 6**.
3. In child B, the first leaf examined is 4. MIN can therefore force a value of **at most 4** at B, so β = 4.
4. Since β = 4 ≤ α = 6, MAX will never choose B (A is already better), regardless of what the other leaves under B are. They are **pruned** without being evaluated.

Effect on complexity

Case	Nodes examined
Plain minimax	O(b^d)
Alpha-beta, worst ordering (nothing pruned)	O(b^d)
Alpha-beta, average ordering	about O(b^(3d/4))
Alpha-beta, perfect ordering (best moves first)	O(b^(d/2))

With best-case ordering, the AI can search **about twice as deep** in the same time. The effectiveness depends on **move ordering**, improved by using the previous iteration's best move first, the killer-move and history heuristics, and sorting moves by a quick estimate (captures and attacks first). **Transposition tables** store values of states already seen and avoid re-searching them.

(iv) Design of an Evaluation Function

When the search is cut off at a depth limit, a heuristic **Eval(s)** estimates how favourable the state is for the AI. A common form is a weighted linear combination:

$Eval(s) = w_1 \cdot f_1(s) + w_2 \cdot f_2(s) + ... + w_n \cdot f_n(s)$

Feature f	What it measures	Why it matters
f_1: Army strength	Total value of own units minus enemy units (health, attack, cost)	Military superiority is the most direct indicator of winning battles.
f_2: Resource economy	Income rate and stockpile of resources, number of workers	Resources fund future units and technology; economic lead gives long-term advantage.
f_3: Map / territory control	Share of key locations, bases, and chokepoints held	Control gives vision, resources, and safer movement.
f_4: Technology and production capacity	Upgrades researched, number of production buildings	Predicts the ability to replace losses and field stronger units.
f_5: Unit positioning and threats	Distance to enemy, formations, units under attack	A well-positioned army beats an equal army in a bad position.
f_6: Base safety (negative weight for risk)	Exposure of the main base and key buildings	Protects against sudden loss of the game.
f_7: Tempo / time	Progress on timing objectives or build-order milestones	Being ahead in tempo allows initiative.

Design points

- Features should be **cheap to compute**, because the function is called at very many leaf nodes.
- Normalise the features to a common scale so that no feature dominates by magnitude.
- Weights reflect importance (for example army strength and base safety highest) and are **tuned** through self-play, human expert knowledge, or machine learning.
- Winning and losing terminal states receive very large positive and negative values so that they override the heuristic estimates.

- Add a **quiescence check**: do not evaluate in the middle of a battle; search a little deeper until the situation is stable, to reduce the horizon effect.

(v) Strategy for Balancing Optimality and Real-Time Performance

Recommended strategy: depth-limited alpha-beta search with iterative deepening, move ordering, and transposition tables, driven by a time budget, and supported by action abstraction.

1. **Iterative deepening.** Search to depth 1, then 2, then 3, and so on until the time limit is reached; always keep the best move from the last completed depth. This makes the algorithm **anytime**: a valid decision is available whenever time runs out.
2. **Time management.** Allocate more time to critical moments (battles, expansion decisions) and less to routine moves.
3. **Move ordering and pruning.** Examine promising moves first so alpha-beta prunes more; optionally use **forward pruning** (beam search), keeping only the top few moves per node.
4. **Action abstraction.** Group unit actions into macro moves to reduce the branching factor.
5. **Transposition tables** to avoid repeated work and use of **parallel threads** for separate subtrees.
6. **Good evaluation function** with quiescence search to make shallow search reliable.
7. **Alternative for very large or uncertain games:** Monte Carlo Tree Search (MCTS) can be combined with the evaluation function, since it handles huge branching factors and imperfect information and can stop at any time.

Justification. Full minimax gives perfect optimality only if the entire tree is searched, which is impossible in real time. Alpha-beta keeps the same optimal result within the depth searched while pruning most of the work; iterative deepening and a quality evaluation function guarantee a good move within the deadline. The result is a **near-optimal decision that always arrives on time**.

Conclusion

Each scenario calls for a different family of AI techniques: **A* with incremental replanning** for dynamic route planning (Scenario 1), **local search meta-heuristics (GA and SA)** for large optimisation (Scenario 2), **online search** for unknown environments (Scenario 3), **CSP with backtracking and constraint propagation** for scheduling (Scenario 4), and **alpha-beta pruning with a good evaluation function** for adversarial real-time games (Scenario 5). The best choice always depends on how much is known, how fast decisions are needed, and the cost of being wrong.